

DOCUMENTO TÉCNICO DE ARQUITECTURA

*Prototipo de Aplicación Web de Gestión de Lecciones Aprendidas en Proyectos
HSEQ Potenciada con Inteligencia Artificial*

Anexo técnico del trabajo de grado

Breidy Alexis Acevedo Díaz

Maestría en Gerencia Estratégica de Gobierno de TI

Universidad Manuela Beltrán (UMB) — Facultad de Ingeniería

Bogotá, Colombia — 2026

1. Introducción y propósito

Este documento describe la arquitectura y la implementación del prototipo desarrollado como parte del trabajo de grado, correspondiente a los objetivos específicos OE1 (diseño de la arquitectura) y OE2 (desarrollo del prototipo). Su fin es dejar constancia técnica y auditable de las decisiones de diseño y del funcionamiento del sistema, de modo que la solución sea comprensible y reproducible por terceros.

El prototipo es una aplicación web cuyo alcance se concentra exclusivamente en la funcionalidad de búsqueda de lecciones aprendidas potenciada con inteligencia artificial, dejando fuera funcionalidades genéricas (gestión de usuarios, capacitaciones, planes de acción, etc.). La búsqueda se apoya en procesamiento de lenguaje natural mediante representaciones vectoriales (embeddings) generadas con servicios de Microsoft Azure, lo que permite recuperar registros por significado y no solo por coincidencia literal de palabras.

2. Visión general de la arquitectura

La solución adopta una arquitectura sin servidor (serverless) sobre Microsoft Azure, compuesta por un frontend web, un backend de funciones, una base de datos documental y dos servicios de inteligencia artificial y búsqueda. El flujo general es el siguiente: el usuario formula una consulta en el frontend; el backend convierte esa consulta en un vector mediante el modelo de embeddings; con ese vector se realiza una búsqueda semántica sobre un índice vectorial; y los registros más cercanos se devuelven ordenados por similitud.

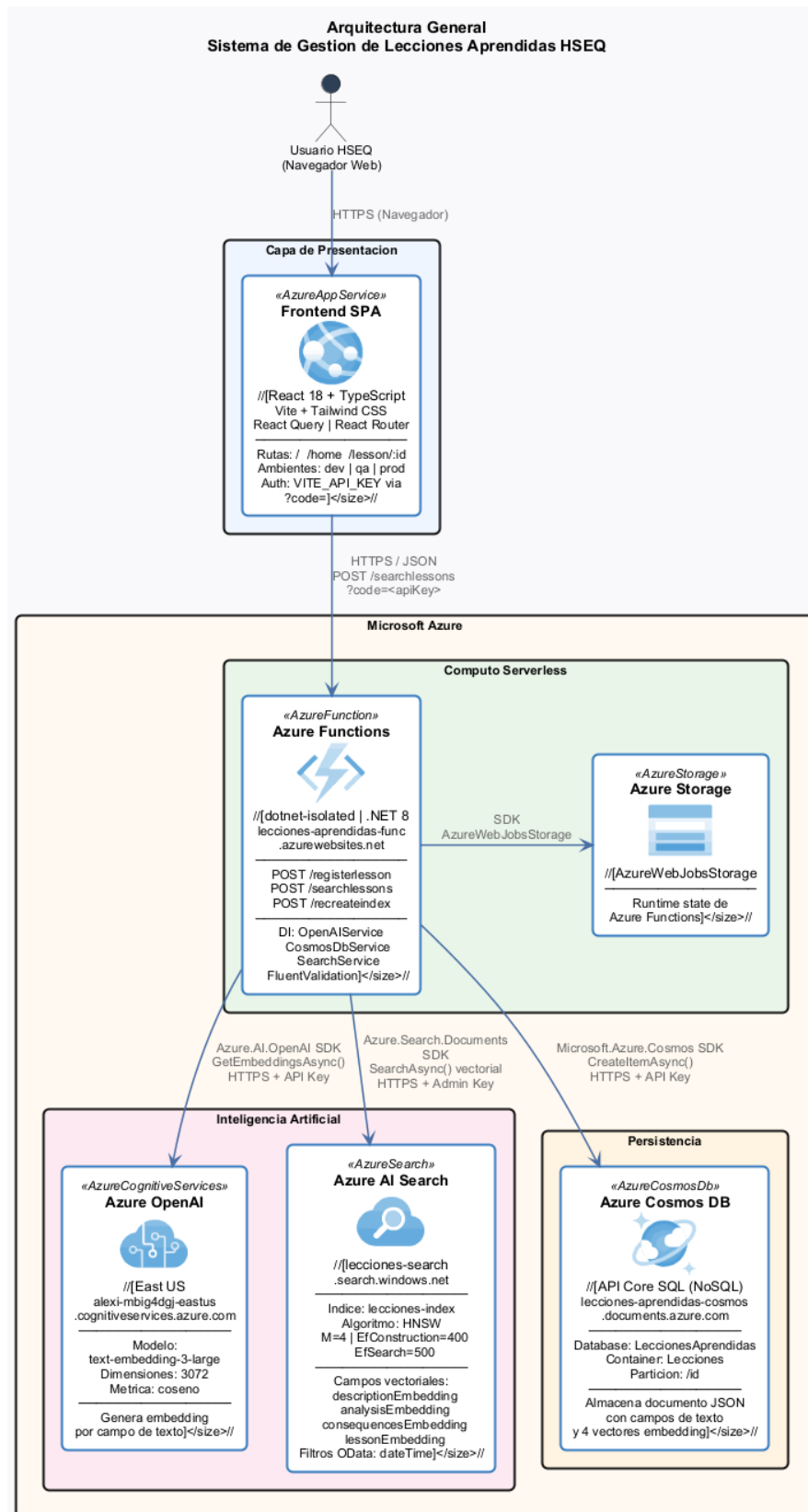


Figura 1. Diagrama de arquitectura general (v2). Fuente: elaboración propia (OE1).

En el nivel más alto de abstracción, el diagrama de contexto (C4 – Nivel 1) sitúa el sistema frente a sus usuarios y los servicios externos de los que depende.

C4 Nivel 1 - Contexto: Sistema de Gestion de Lecciones Aprendidas HSEQ

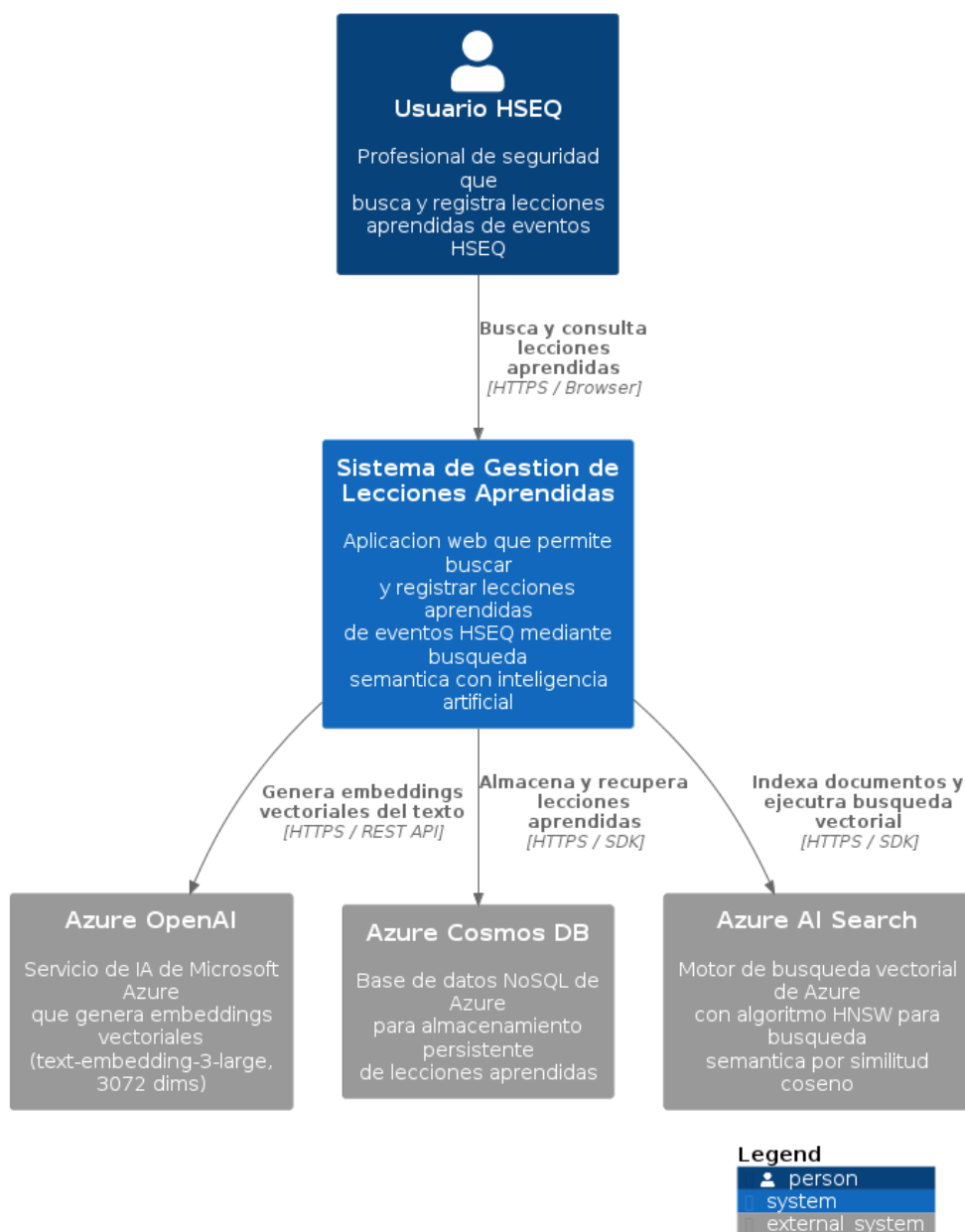


Figura 2. Diagrama de contexto (C4 - Nivel 1). Fuente: elaboración propia (OE1).

Los componentes principales y su responsabilidad se resumen a continuación:

- **Frontend (aplicación web):** interfaz de usuario para formular búsquedas y visualizar los resultados. Desplegado como sitio estático.
- **Backend (API de funciones):** expone los endpoints, valida las solicitudes, genera los embeddings de la consulta y orquesta la búsqueda vectorial y la persistencia.

- **Servicio de embeddings (Azure OpenAI):** convierte texto en vectores numéricos que representan su significado.
- **Índice de búsqueda (Azure AI Search):** almacena los vectores y ejecuta la búsqueda por similitud (vecinos más cercanos).
- **Base de datos documental (Azure Cosmos DB):** almacena las lecciones aprendidas como fuente de verdad.

3. Stack tecnológico

La Tabla 1 resume las tecnologías empleadas, con sus versiones, para efectos de reproducibilidad.

Capa	Tecnología	Versión	Propósito
Frontend	React + TypeScript	React 18.3	Interfaz de usuario (SPA)
Frontend	Vite	—	Empaquetado y servidor de desarrollo
Frontend	TailwindCSS + shadcn/ui (Radix UI)	—	Estilos y componentes de interfaz
Frontend	TanStack React Query	5.x	Gestión de peticiones y caché
Frontend	React Hook Form + Zod	—	Formularios y validación
Backend	Azure Functions (isolated worker)	v4	Cómputo sin servidor
Backend	.NET / C#	net8.0	Lenguaje y runtime del backend
Backend	Azure.AI.OpenAI	1.0.0-beta.12	Ciente de embeddings
Backend	Azure.Search.Documents	11.6.0	Ciente de Azure AI Search
Backend	Microsoft.Azure.Cosmos	3.37.0	Ciente de Cosmos DB
Backend	FluentValidation	12.0.0	Validación de solicitudes
IA	Azure OpenAI — text-embedding-3-large	3072 dim.	Generación de embeddings
Búsqueda	Azure AI Search (índice vectorial)	HNSW / coseno	Búsqueda semántica
Datos	Azure Cosmos DB (NoSQL)	—	Persistencia de lecciones
Despliegue	Azure Static Web Apps / Azure Function App	—	Hospedaje de frontend y backend
CI/CD	GitHub Actions	—	Integración y despliegue continuos

4. Componentes del backend

El backend está construido como una aplicación de Azure Functions (.NET 8, modelo aislado) e inyecta sus dependencias mediante el contenedor de servicios. Al iniciar, la aplicación verifica y crea el índice de búsqueda si no existe. Se organiza en tres capas:

- **Functions:** los endpoints HTTP que reciben las solicitudes (registro, consulta y búsqueda de lecciones, reindexación y sugerencias).
- **Services:** la lógica de negocio, dividida en tres servicios inyectables: OpenAIService (genera embeddings), SearchService (crea el índice y ejecuta la búsqueda vectorial) y CosmosDbService (persistencia).
- **Models:** las entidades y los objetos de solicitud/respuesta (LessonLearned, CreateLessonRequest, SearchLessonRequest, resultados paginados, etc.), junto con las opciones de configuración de cada servicio.

5. Modelo de datos

Cada lección aprendida se modela con la entidad LessonLearned. Además de los campos de contenido, la entidad mantiene un campo combinado (searchContent) y cinco vectores de embedding, uno por cada campo indexable, lo que permite dirigir la búsqueda semántica a un campo específico. La Tabla 2 describe los campos principales.

Campo	Tipo	Descripción
id	string	Identificador único (GUID).
code	string	Código único del evento.
description	string	Descripción detallada de lo ocurrido.
situationType	string	Tipo de situación (near miss, incidente, acto inseguro, etc.).
location	string	Ubicación del evento.
relatedPosition	string	Cargo de la persona involucrada o reportante.
analysis	string	Análisis de causas del evento.
consequences	string	Consecuencias reales o potenciales.
lesson	string	Aprendizaje generado.
dateTime	fecha/hora	Fecha del evento.
source	entero	Fuente del registro (Anyi = 1, Kimy = 2).
searchContent	string	Concatenación de los campos relevantes; base del embedding principal.
*Embedding (x5)	vector (3072)	Embeddings de searchContent, description, analysis, consequences y lesson.

6. Motor de búsqueda semántica

Esta es la funcionalidad central del prototipo y el componente que aporta la inteligencia artificial. Su funcionamiento, descrito para efectos de reproducibilidad, es el siguiente:

Indexación. Al registrar una lección, el backend genera con Azure OpenAI (modelo text-embedding-3-large, 3072 dimensiones) los vectores de embedding de sus campos y los almacena en un índice de Azure AI Search. El índice define, además de los campos textuales, cinco campos vectoriales configurados con un perfil de búsqueda vectorial basado en el algoritmo HNSW y métrica de similitud del coseno.

Consulta. Cuando el usuario busca, el backend toma el texto de la consulta y genera su embedding con el mismo modelo. Con ese vector ejecuta una búsqueda de vecinos más cercanos (KNN) sobre el campo vectorial seleccionado (por defecto, el de searchContent). Opcionalmente aplica filtros por rango de fechas y por fuente, un umbral de puntuación mínima (minScore) y paginación. Cada resultado se devuelve con una puntuación de similitud entre 0 y 1.

Parámetros del índice vectorial (HNSW): métrica coseno; dimensión de los vectores 3072. La consulta recupera hasta un máximo de candidatos, sobre los que se aplica el umbral de similitud y luego la paginación.

Nota sobre el comportamiento de los resultados: si no se especifica un umbral (minScore), el motor devuelve siempre los registros más cercanos ordenados por similitud, sin descartar ninguno; el parámetro minScore permite fijar un corte de relevancia cuando se desee.

7. API REST

El backend expone los siguientes endpoints. Todos requieren clave de función (AuthorizationLevel.Function), es decir, no son de acceso anónimo. La Tabla 3 los resume.

Endpoint	Método	Propósito
RegisterLesson	POST	Registra una lección: genera sus embeddings y la guarda en Cosmos DB y en el índice.
GetLesson (lessons/{id})	GET	Obtiene una lección por su identificador.
SearchLessons	POST	Búsqueda semántica vectorial con filtros, umbral y paginación.
SuggestLessons	POST	Autocompletado con resaltado de coincidencias.
ReindexFromCosmosDb	POST	Reconstruye el índice a partir de Cosmos DB.
RecreateIndex	POST	Recrea la definición del índice de búsqueda.

La solicitud de búsqueda (SearchLessons) admite: query (texto libre, mínimo 3 caracteres), field (campo vectorial: searchContent, description, analysis, consequences o lesson), dateFrom / dateTo, minScore (0 a 1), source, pageNumber y pageSize (10, 25, 50 o 100).

8. Frontend

El frontend es una aplicación de página única (SPA) construida con React 18 y TypeScript, empaquetada con Vite y estilizada con TailwindCSS y componentes de shadcn/ui (sobre Radix UI). La gestión de peticiones a la API se realiza con TanStack React Query, y los formularios con React Hook Form y validación con Zod.

La Figura 3 muestra la interfaz de búsqueda del prototipo: el panel lateral derecho permite introducir el contexto de la consulta en lenguaje natural y el área central presenta los resultados recuperados.

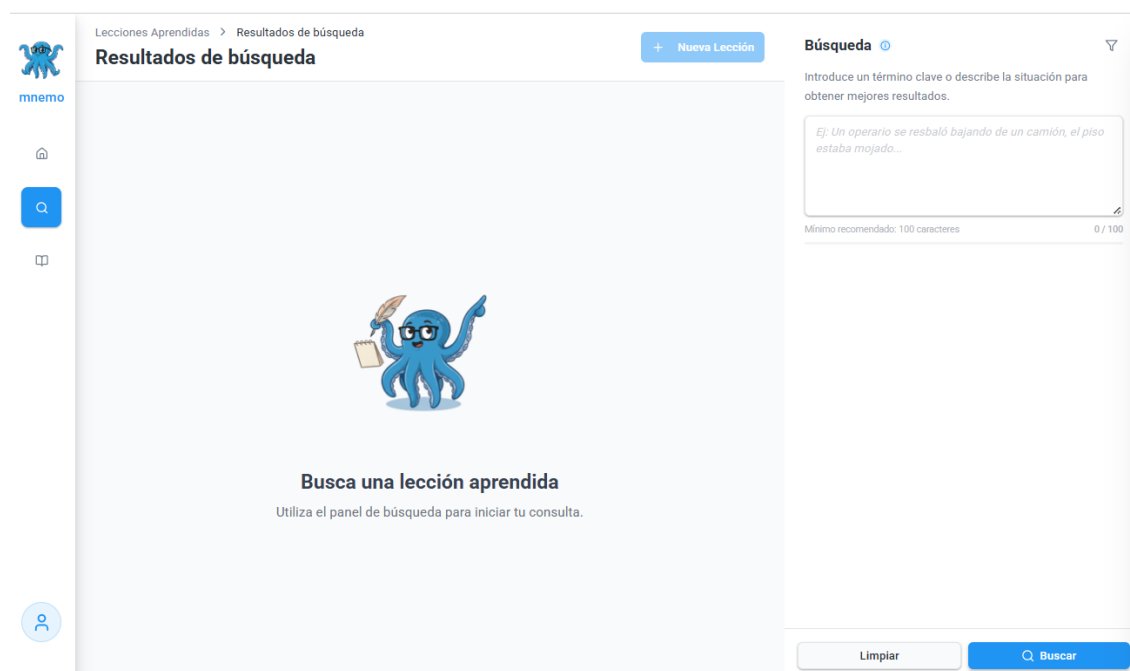


Figura 3. Interfaz de búsqueda del prototipo. Fuente: elaboración propia.

La configuración se realiza mediante variables de entorno con prefijo VITE_: VITE_API_URL (URL base de la API), VITE_API_KEY (clave de la API), VITE_ENV (ambiente: dev, qa o production) y VITE_APP_TITLE. Esto permite apuntar el mismo código a distintos ambientes sin recompilar la lógica.

9. Despliegue e infraestructura

El sistema se despliega íntegramente en Microsoft Azure y su publicación se automatiza con GitHub Actions:

- **Frontend:** Azure Static Web Apps. El flujo de CI/CD compila el proyecto (salida en dist) y lo publica en cada cambio.
- **Backend:** Azure Function App (lecciones-aprendidas-func), con despliegue continuo desde GitHub Actions. Registra telemetría mediante Application Insights.
- **Servicios gestionados:** Azure OpenAI (embeddings), Azure AI Search (índice vectorial) y Azure Cosmos DB (persistencia).
- **Aplicativo desplegado:** <https://salmon-coast-08a22710f.6.azurestaticapps.net/>
- **Código fuente:** backend en <https://github.com/b1diaz/lis-backend> y frontend en <https://github.com/b1diaz/lis-frontend>. Los repositorios se mantienen públicos de forma temporal mientras el jurado realiza la evaluación; una vez finalizada, se restablecerá su acceso privado.

10. Configuración y seguridad

La configuración del backend se agrupa en tres secciones, cuyos valores se administran como secretos y no se versionan en el repositorio (el archivo local de configuración está excluido del control de versiones):

Sección	Claves	Propósito
OpenAI	Endpoint, Key, EmbeddingDeployment	Servicio de embeddings (modelo text-embedding-3-large).
AzureSearch	Endpoint, AdminKey, IndexName, CognitiveServicesKey	Índice de búsqueda vectorial.
CosmosDb	Endpoint, Key, DatabaseName, ContainerName	Base de datos documental.

Consideraciones de seguridad: los endpoints requieren clave de función, por lo que el API no es de acceso anónimo. Las credenciales de los servicios se manejan como secretos fuera del control de versiones. Como buena práctica para un entorno productivo, se recomienda migrar el manejo de secretos a Azure Key Vault o identidades administradas (Managed Identity), y definir políticas de residencia y retención de datos acordes con la normativa aplicable (Ley 1581 de 2012), dado que los registros pueden contener datos personales.

11. Reproducibilidad

Para reconstruir el prototipo se requiere una suscripción de Azure con los siguientes recursos y pasos:

- Aprovisionar Azure OpenAI y desplegar el modelo de embeddings text-embedding-3-large.
- Crear un servicio de Azure AI Search (el índice vectorial se crea automáticamente al iniciar el backend, con campos de 3072 dimensiones, perfil HNSW y métrica coseno).
- Crear una base de datos y un contenedor en Azure Cosmos DB.
- Configurar las tres secciones de credenciales (OpenAI, AzureSearch, CosmosDb) descritas en la sección 10.
- Publicar el backend en Azure Functions (.NET 8) y el frontend en Azure Static Web Apps, configurando las variables VITE_ del frontend.
- Cargar las lecciones mediante el endpoint RegisterLesson (o ReindexFromCosmosDb), lo que genera y almacena los embeddings.

12. Índice de diagramas de arquitectura (OE1)

El conjunto completo de diagramas que sustenta el diseño se elaboró en PlantUML. La Tabla 4 los relaciona.

Diagrama	Nivel / tipo	Qué representa
C4 – Nivel 1	Contexto	El sistema y sus usuarios y sistemas externos.
C4 – Nivel 2	Contenedor	Frontend, backend, base de datos y servicios de Azure.
C4 – Nivel 3	Componente	Módulos internos del backend (Functions, Services, Models).
Diagrama de flujo	Proceso (v1 y v2)	El proceso de búsqueda de lecciones por el usuario.
Diagrama de secuencia	Interacción	Intercambio entre usuario, frontend, backend y servicios de IA durante una búsqueda.
Arquitectura general	Despliegue (v1 y v2)	Vista de alto nivel de la infraestructura en Azure (v2 con iconos oficiales).

Se conservan las versiones definitivas de cada diagrama; las versiones preliminares se mantienen solo con fines de trazabilidad.

El conjunto completo de diagramas, en formato de imagen y con sus fuentes en PlantUML, se incluye en la carpeta Diagramas que acompaña a este anexo.